

DOSSIER TECHNIQUE

PROJET : SYSTÈME DE GESTION DE DONNÉES SÉCURISÉ (SGDS) / CLOUD PRIVÉ

FORMATION : YNOV INFORMATIQUE - BACHELOR 1 (UF INFRA)

I. PRÉSENTATION DU PROJET

1.1 Contexte et Problématique

Dans le cadre de l'unité de formation **INFRA**, ce projet vise à répondre à une problématique de mobilité et de sécurité des données. Une entreprise souhaite que ses collaborateurs accèdent à un espace de stockage centralisé, de n'importe où, tout en garantissant que les données ne soient pas exposées sur l'internet public. L'enjeu majeur est de construire une infrastructure capable de gérer la redondance des données (backup) et la sécurité des accès de manière totalement autonome.

1.2 La Solution Retenue : Architecture Hybride

Nous avons opté pour une architecture **client-serveur hybride**. Initialement conçu sur un serveur physique (**Raspberry Pi / NAS**) pour simuler un centre de données local, le système interagit avec des machines virtuelles (VMs) simulant les postes de travail distants. Pour la présentation finale, le système a été entièrement répliqué en environnement virtualisé pour garantir une démonstration fluide tout en conservant la segmentation réseau initiale.

II. MÉTHODOLOGIE ET PHASES DE RÉALISATION

2.1 Étape 1 : Préparation de l'environnement "Host"

Le premier défi technique a été l'installation d'**Ubuntu Server** sur une architecture **ARM (Raspberry Pi)**. Cela a nécessité :

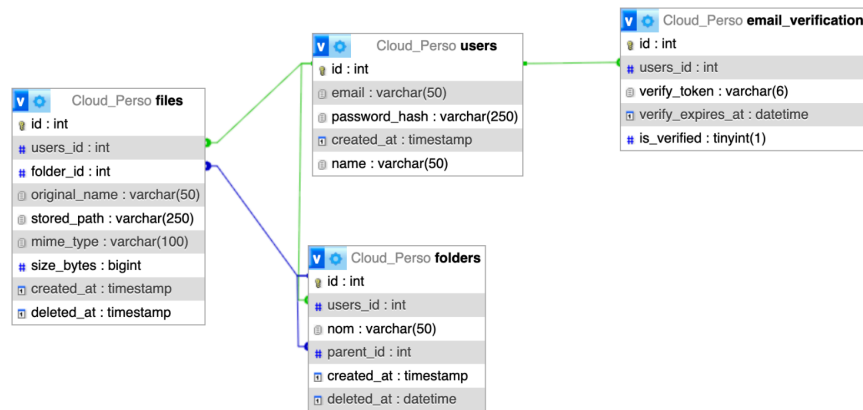
- La gestion spécifique du flashage de l'image système.
- L'administration exclusive via **SSH** pour simuler une gestion de serveur "Headless" (sans écran).

2.2 Étape 2 : Ingénierie Logicielle (Le défi Go & MySQL)

C'est le cœur applicatif du projet. Nous avons développé une solution sur mesure :

- **Conception de la DB (MySQL)** : Nous avons modélisé une base de données simulant un système de fichiers. Le choix du **parent_id** pour gérer l'arborescence virtuelle a été un défi algorithmique majeur : cela permet de naviguer dans des dossiers sans les créer physiquement sur le disque, optimisant ainsi les performances.

Voici un screen de notre base de données depuis phpmyadmin :



- **Développement du Backend (Go)** : Nous avons codé les fonctions de transfert (Upload/Download) en veillant à la gestion des **buffers** (tampons) pour ne pas saturer la RAM du Raspberry Pi lors de transferts importants.

Voici le lien vers le github du projet afin de voir le code si besoin :

https://github.com/Mayel-0/Cloud_perso

- **Sécurisation Applicative** : Intégration d'un système de hashage pour les mots de passe, gestion du **Soft Delete** (colonne **deleted_at** pour la corbeille) et module **SMTP** pour l'envoi de codes **A2F** (Authentification à deux facteurs).

2.3 Étape 3 : Mise en place de la forteresse réseau

Une fois le service fonctionnel, nous avons sécurisé les accès :

- **Configuration WireGuard (VPN)** : Génération des paires de clés (publique/privée) pour chaque utilisateur autorisé.
 - **Scripting UFW** : Configuration d'un pare-feu restrictif en mode "Deny All". Seuls les flux provenant du tunnel VPN sont autorisés à interroger l'application.
-

III. ARCHITECTURE DÉTAILLÉE

3.1 La Couche Matérielle (Le Serveur)

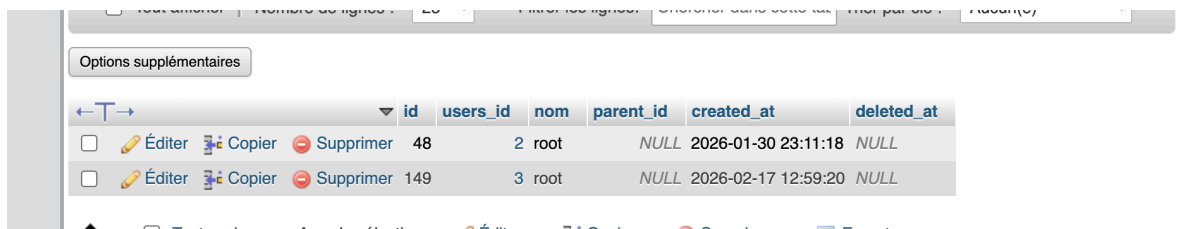
- **Hôte** : Raspberry Pi / Instance virtuelle (Ubuntu Server 22.04 LTS).
- **Stockage Multi-disque** : Le système (OS) est isolé sur le support primaire (Carte SD), tandis que les données utilisateurs sont montées sur un volume de stockage dédié. Cette isolation garantit la survie des données en cas de crash du système d'exploitation.

3.2 La Couche Applicative (Le Service)

- **Backend (Golang)** : Le service web est lancé via la commande `go run main.go` (en développement) ou via un binaire compilé. **Note importante** : Nous n'utilisons pas d'hébergeur ou de serveur tiers comme **Nginx**. L'application Go intègre son propre serveur HTTP natif. C'est via le VPN que l'on accède directement à ce service.
- **Base de données (MySQL)** : Créée initialement sur un environnement de développement (MAMP), la structure a été exportée en SQL puis injectée via SSH sur le serveur de production. Les variables d'environnement ont été adaptées pour lier le code Go à l'instance MySQL.
- **Interface (HTML/CSS)** : Une interface web moderne permet de lister, uploader et télécharger les fichiers, rendant le système accessible à un utilisateur non-technique.

3.3 Gestion des Données et Redondance

- **Arborescence Virtuelle** : La DB contient toute la hiérarchie. Si un dossier est créé dans la racine (`root`), il obtient le `parent_id` de la racine.



	id	users_id	nom	parent_id	created_at	deleted_at
☐ Éditer Copier Supprimer	48	2	root	NULL	2026-01-30 23:11:18	NULL
☐ Éditer Copier Supprimer	149	3	root	NULL	2026-02-17 12:59:20	NULL

- **Stockage Physique** : Le système crée un sous-dossier physique par utilisateur. Les fichiers y sont stockés avec une gestion de backup automatisée.

IV. MISE EN ŒUVRE TECHNIQUE ET AUTOMATISATION

4.1 Stratégie de Backup Automatisée

Nous avons conçu un script **Bash** sur mesure, déclenché par une tâche **Cron**.

- **Fonctionnement** : Le script vérifie le point de montage du disque de sauvegarde, puis utilise `rsync` pour répliquer les données modifiées.
- **Sécurité** : En cas d'échec, un log d'erreur est généré pour le diagnostic.

4.2 Transition Physique vers Virtuel (Démonstration)

Pour la présentation, nous avons réalisé une image disque du Raspberry Pi convertie en Machine Virtuelle (VM). Ce processus a nécessité une réadaptation réseau importante : passage des interfaces `eth0` (physique) vers `ens33` (virtuel) pour maintenir la connectivité du VPN et de la base de données.

V. WORKFLOW : CHEMINEMENT D'UNE REQUÊTE

1. **Accès Initial** : L'utilisateur tente d'accéder à <http://cloud.local:8080>. La connexion échoue (bloquée par UFW).
 2. **Tunneling** : L'utilisateur active son client **WireGuard** avec une configuration valide.
 3. **Authentication** : Le serveur reconnaît l'IP virtuelle. L'interface HTML s'affiche.
 4. **Transaction** : L'utilisateur ajoute un fichier. Le code Go écrit le fichier sur le disque de données **ET** sur le disque de backup, tout en mettant à jour la base MySQL.
-

VI. CONCLUSION ET PERSPECTIVES

Ce projet démontre qu'une infrastructure de stockage n'est pas qu'une question de disque dur, mais un écosystème mêlant réseau, sécurité et développement. Nous avons réussi à créer une solution "Zero Trust" où seul le tunnel VPN permet l'accès aux données. Une évolution possible serait la conteneurisation via **Docker** pour simplifier encore davantage le déploiement du couple Go/MySQL.